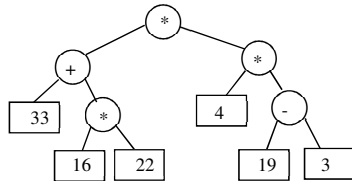


EXAMEN D'ALGORITHMIQUE III L3I

Exercice 1

On considère les expressions arithmétiques n'utilisant que les opérateurs binaires $\{+, -, *, /\}$; ces expressions peuvent être représentées par des arbres binaires dont les nœuds sont étiquetés par l'un des quatre opérateurs tandis que les feuilles sont étiquetées par des opérandes (des entiers).
L'arbre binaire ci-dessous représente l'expression arithmétique suivante $((33+(16*22))*(4*(19-3)))$



1) Ecrire **en C**, les structures de données nécessaire pour représenter les expressions arithmétiques.

En respectant la structure; écrire en langage algorithmique ou en C les algorithmes 2 à 4 :

- Déterminer le nombre d'opérandes présent dans une expression arithmétique donnée.
- Déterminer l'opérateur qui y est le plus présent dans une expression arithmétique donnée.
- Vérifier si un arbre binaire donné contenant que des opérateurs binaires et des opérandes (entiers par exemple) est bien valide.
- Proposer **en C** une structure de donnée représentant les expressions arithmétiques d'une manière générique.

Exercice 2

- Dessinez tous les AVL qui contiennent les valeurs 1, 2, 3, 4, 5.
- Insérer dans l'AVL ci-contre les valeurs 18 puis 60.
- Supprimer dans l'AVL ci-contre la valeur 50.
- Ecrire en langage algorithmique ou en C, l'insertion d'un élément donné dans un AVL donné (note : n'oublier pas de gérer l'équilibrage de l'AVL).



Exercice 3

Il existe différentes façons d'implémenter les arbres généraux :

A) Lorsque le nombre de fils n'est pas connu à priori, on utilise une liste chaînée de fils.

```
typedef struct cellule *ListArbres ;
typedef struct noeud *ArbreG;
struct noeud{
    Telement donnee;
    ListArbres fils ;
};
ArbreG premierfils;
ListArbres suivant;
```

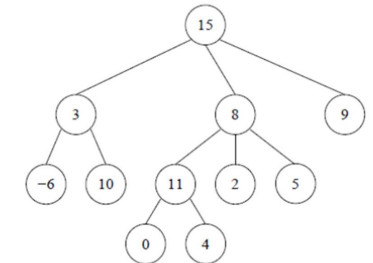
B) Forme condensée avec une seule structure : un lien vers le fils aîné et un lien vers le frère

```
typedef struct noeud * ArbreG;
```

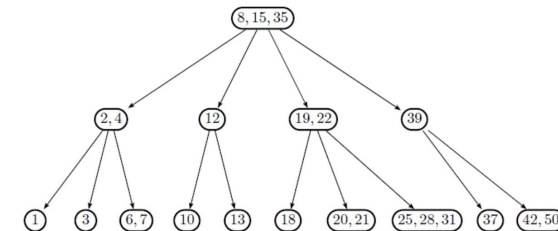
```
struct noeud{
    Telement donnee;
    ArbreG filsAine;
    ArbreG freres;
};
```

Pour chacune des représentations définies ci-dessus :

- Ecrire **en C**, la génération de l'arbre ci-contre.
- Ecrire en langage algorithmique ou en C, le degré d'un arbre donné.



Exercice 4



- Insérer l'élément 30 dans l'arbre 2-3-4 ci-dessus.
- Ecrire **en C**, les structures de données nécessaire pour représenter les arbres 2-3-4 dont le membre donné est de type TElement (Entier par exemple).
- Déterminer en langage algorithmique ou en C, l'adresse d'un élément donné dans un arbre 2-3-4 donné.
- Ecrire en langage algorithmique ou en C, le parcours infixe d'un arbre 2-3-4 donné (rappel : le parcours infixe dans un arbre 2-3-4 donne une liste ordonnée des valeurs)

Exercice 5

- Décrire uniquement le principe de la résolution des collisions avec les méthodes suivantes :

- Hachage coalescent sans réserve.
- Hachage linéaire.

- Application : soit les éléments E1, E2, E3, E4, E5, E6, E7, E8, E9 dont les valeurs de hachage respectivement sont : 1, 5, 2, 5, 1, 8, 11, 9, 5, 10.

On suppose que la table de hachage commence à l'indice 1 et de taille M=11.

Décrire graphiquement pour chacune des méthodes (a) et (b) le remplissage de la table de hachage.

- Pour supprimer un élément dans une table de hachage, nous marquons sa place comme libre. Une place de la table possède donc 3 états possible : vide, occupé, libéré.
 - Ecrire **en C**, la structure de données représentant une table de hachage dont le membre information est de type TElement.
 - En respectant la structure de données ; écrire en langage algorithmique ou en C ; la recherche d'un élément donné dans la table de hachage de votre choix (méthode a) ou b)).